



(2024-05-22) 前端高频算法原题解析 & 从面试官角度剖析算法面试

1. 基础--时间复杂度&空间复杂度

复杂度分析是整个算法学习的精髓，只要掌握了它，数据结构和算法的内容基本上就掌握了一半了。

什么是复杂度分析？

数据结构和算法解决是“如何让计算机更快时间、更省空间的解决问题”。因此需从执行时间和占用空间两个维度来评估数据结构和算法的性能。分别用时间复杂度和空间复杂度两个概念来描述性能问题，二者统称为复杂度。

复杂度描述的是算法执行时间（或占用空间）与数据规模的增长关系。

为什么要进行复杂度分析？

和性能测试相比，复杂度分析有不依赖执行环境、成本低、效率高、易操作、指导性强的特点。掌握复杂度分析，将能编写出性能更优的代码，有利于降低系统开发和维护成本。

如何进行复杂度分析？

大 O 表示法

算法的执行时间与每行代码的执行次数成正比，用 $T(n) = O(f(n))$ 表示，其中 $T(n)$ 表示算法执行总时间， $f(n)$ 表示每行代码执行总次数，而 n 往往表示数据的规模。这就是大 O 时间复杂度表示法。

1.1 时间复杂度

1.1.1 定义

算法的时间复杂度，也就是算法的时间量度。

大 O 时间复杂度表示法 实际上并不具体表示代码真正的执行时间，而是表示 **代码执行时间随数据规模增长的变化趋势**，所以也叫 **渐进时间复杂度**，简称 **时间复杂度** (asymptotic time complexity)。

例子1:

```
1 function aFun() {  
2     console.log("Hello, World!");    // 需要执行 1 次  
3     return 0;        // 需要执行 1 次  
4 }
```

那么这个方法需要执行 2 次运算。

例子 2:

```
1 function bFun(n) {  
2     for(let i = 0; i < n; i++) {      // 需要执行 (n + 1) 次  
3         console.log("Hello, World!"); // 需要执行 n 次  
4     }  
5     return 0;        // 需要执行 1 次  
6 }
```

那么这个方法需要执行 $(n + 1 + n + 1) = 2n + 2$ 次运算。

例子 3:

```
1 function cal(n) {  
2     let sum = 0; // 1 次  
3     let i = 1;  // 1 次  
4     let j = 1;  // 1 次  
5     for (; i <= n; ++i) { // n 次  
6         j = 1; // n 次  
7         for (; j <= n; ++j) { // n * n , 也即是 n平方次  
8             sum = sum + i * j; // n * n , 也即是 n平方次  
9         }  
10    }  
11 }
```

注意，这里是二层 for 循环，所以第二层执行的是 $n * n = n(2)$ 次，而且这里的循环是 ++i，和例子 2 的是 i++，是不同的，是先加与后加的区别。

那么这个方法需要执行 $(n(2) + n(2) + n + n + 1 + 1 + 1) = 2n(2) + 2n + 3$ 。

1.1.2 特点

以时间复杂度为例，由于 **时间复杂度** 描述的是算法执行时间与数据规模的 **增长变化趋势**，所以 **常量、低阶、系数** 实际上对这种增长趋势不产生决定性影响，所以在做时间复杂度分析时 **忽略** 这些项。

所以，上面例子1的时间复杂度为 $T(n) = O(1)$ ，例子2的时间复杂度为 $T(n) = O(n)$ ，例子3的时间复杂度为 $T(n) = O(n(2))$ 。

1.2 时间复杂度分析

1. 只关注循环执行次数最多的一段代码

单段代码看高频：比如循环。

```
1 function cal(n) {
2   let sum = 0;
3   let i = 1;
4   for (; i <= n; ++i) {
5     sum = sum + i;
6   }
7   return sum;
8 }
```

执行次数最多的是 for 循环及里面的代码，执行了 n 次，所以时间复杂度为 $O(n)$ 。

2. 加法法则：总复杂度等于量级最大的那段代码的复杂度

多段代码取最大：比如一段代码中有单循环和多重循环，那么取多重循环的复杂度。

```
1 function cal(n) {
2   let sum_1 = 0;
3   let p = 1;
4   for (; p < 100; ++p) {
5     sum_1 = sum_1 + p;
6   }
7
8   let sum_2 = 0;
9   let q = 1;
10  for (; q < n; ++q) {
11    sum_2 = sum_2 + q;
```

```

12     }
13
14     let sum_3 = 0;
15     let i = 1;
16     let j = 1;
17     for (; i <= n; ++i) {
18         j = 1;
19         for (; j <= n; ++j) {
20             sum_3 = sum_3 + i * j;
21         }
22     }
23
24     return sum_1 + sum_2 + sum_3;
25 }

```

上面代码分为三部分，分别求 sum_1、sum_2、sum_3，主要看循环部分。

第一部分，求 sum_1，明确知道执行了 100 次，而和 n 的规模无关，是个常量的执行时间，不能反映**增长变化趋势**，所以时间复杂度为 $O(1)$ 。

第二和第三部分，求 sum_2 和 sum_3，时间复杂度是和 n 的规模有关的，分别为 $O(n)$ 和 $O(n^2)$ 。

所以，取三段代码的最大量级，上面例子的最终的时间复杂度为 $O(n^2)$ 。

同理类推，如果有 3 层 for 循环，那么时间复杂度为 $O(n^3)$ ，4 层就是 $O(n^4)$ 。

所以，**总的时间复杂度就等于量级最大的那段代码的时间复杂度。**

3. 乘法法则：嵌套代码的复杂度等于嵌套内外代码复杂度的乘积

嵌套代码求乘积：比如递归、多重循环等。

```

1 function cal(n) {
2     let ret = 0;
3     let i = 1;
4     for (; i < n; ++i) {
5         ret = ret + f(i); // 重点为 f(i)
6     }
7 }
8
9 function f(n) {
10    let sum = 0;
11    let i = 1;
12    for (; i < n; ++i) {
13        sum = sum + i;
14    }
15    return sum;

```

```
16 }
```

方法 cal 循环里面调用 f 方法，而 f 方法里面也有循环。

所以，整个 cal() 函数的时间复杂度就是， $T(n) = T1(n) * T2(n) = O(n*n) = O(n(2))$ 。

4. 多个规模求加法：比如方法有两个参数控制两个循环的次数，那么这时就取二者复杂度相加

```
1 function cal(m, n) {
2   let sum_1 = 0;
3   let i = 1;
4   for (; i < m; ++i) {
5     sum_1 = sum_1 + i;
6   }
7
8   let sum_2 = 0;
9   let j = 1;
10  for (; j < n; ++j) {
11    sum_2 = sum_2 + j;
12  }
13
14  return sum_1 + sum_2;
15 }
```

以上代码也是求和，求 sum_1 的数据规模为 m、求 sum_2 的数据规模为 n，所以时间复杂度为 $O(m+n)$ 。

公式： $T1(m) + T2(n) = O(f(m) + g(n))$ 。

5. 多个规模求乘法：比如方法有两个参数控制两个循环的次数，那么这时就取二者复杂度相乘

```
1 function cal(m, n) {
2   let sum_3 = 0;
3   let i = 1;
4   let j = 1;
5   for (; i <= m; ++i) {
6     j = 1;
7     for (; j <= n; ++j) {
8       sum_3 = sum_3 + i * j;
9     }
10  }
11 }
```

以上代码也是求和，两层 for 循环，求 sum_3 的数据规模为 m 和 n，所以时间复杂度为 $O(m*n)$ 。

公式： $T1(m) * T2(n) = O(f(m) * g(n))$ 。

1.3 常用的时间复杂度分析

1. 多项式阶：随着数据规模的增长，算法的执行时间和空间占用，按照多项式的比例增长。

包括 $O(1)$ （常数阶）、 $O(\log n)$ （对数阶）、 $O(n)$ （线性阶）、 $O(n \log n)$ （线性对数阶）、 $O(n^2)$ （平方阶）、 $O(n^3)$ （立方阶）。

除了 $O(\log n)$ 、 $O(n \log n)$ ，其他的都可从上面的几个例子中看到。

下面举例说明 **$O(\log n)$ （对数阶）**：

```
1 let i=1;
2 while (i <= n) {
3     i = i * 2;
4 }
```

代码是从 1 开始，每次循环就乘以 2，当大于 n 时，循环结束。

其实就是高中学过的等比数列，i 的取值就是一个等比数列。在数学里面是这样子的：

$2(0) 2(1) 2(2) \dots 2(k) \dots 2(x) = n$

所以，我们只要知道 x 值是多少，就知道这行代码执行的次数了，通过 $2^x = n$ 求解 x，数学中求解得 $x = \log(2)^n$ 。所以上面代码的时间复杂度为 $O(\log(2)^n)$ 。

实际上，不管是以 2 为底、以 3 为底，还是以 10 为底，我们可以把所有对数阶的时间复杂度都记为 $O(\log n)$ 。为什么呢？

因为对数之间是可以互相转换的， $\log_3^n = \log(3)^2 * \log(2)^n$ ，所以 $O(\log(3)^n) = O(C * \log(2)^n)$ ，其中 $C = \log(3)^2$ 是一个常量。

由于 **时间复杂度** 描述的是算法执行时间与数据规模的 **增长变化趋势**，所以 **常量、低阶、系数** 实际上对这种增长趋势不产生决定性影响，所以在做时间复杂度分析时 **忽略** 这些项。

因此，在对数阶时间复杂度的表示方法里，我们忽略对数的“底”，统一表示为 $O(\log n)$ 。

下面举例说明 **$O(n \log n)$ （对数阶）**：

```
1 function aFun(n){
2     let i = 1;
3     while (i <= n) {
```

```

4      i = i * 2;
5  }
6  return i
7  }
8
9  function cal(n) {
10     let sum = 0;
11     for (let i = 1; i <= n; ++i) {
12         sum = sum + aFun(n);
13     }
14     return sum;
15 }

```

aFun 的时间复杂度为 $O(\log n)$ ，而 cal 的时间复杂度为 $O(n)$ ，所以上面代码的时间复杂度为 $T(n) = T_1(\log n) * T_2(n) = O(\log n * n) = O(n \log n)$ 。

2. 非多项式阶：随着数据规模的增长，算法的执行时间和空间占用暴增，这类算法性能极差。

包括 $O(2^n)$ （指数阶）、 $O(n!)$ （阶乘阶）。

$O(2^n)$ （指数阶）例子：

```

1 aFunc( n ) {
2     if (n <= 1) {
3         return 1;
4     } else {
5         return aFunc(n - 1) + aFunc(n - 2);
6     }
7 }

```

参考答案：

显然运行次数， $T(0) = T(1) = 1$ ，同时 $T(n) = T(n - 1) + T(n - 2) + 1$ ，这里的 1 是其中的加法算一次执行。

显然 $T(n) = T(n - 1) + T(n - 2)$ 是一个斐波那契数列，通过归纳证明法可以证明，当 $n \geq 1$ 时 $T(n) < (5/3)^n$ ，同时当 $n > 4$ 时 $T(n) \geq (3/2)^n$ 。

所以该方法的时间复杂度可以表示为 $O((5/3)^n)$ ，简化后为 $O(2^n)$ 。

可见这个方法所需的运行时间是以指数的速度增长的。

如果大家感兴趣，可以试下分别用 1，10，100 的输入大小来测试下算法的运行时间，相信大家会感受到时间复杂度的无穷魅力。

1.4 时间复杂度分类

时间复杂度可以分为：

- **最好情况时间复杂度** (best case time complexity)：在最理想的情况下，执行这段代码的时间复杂度。
- **最坏情况时间复杂度** (worst case time complexity)：在最糟糕的情况下，执行这段代码的时间复杂度。
- **平均情况时间复杂度** (average case time complexity)，用代码在所有情况下执行的次数的加权平均值表示。也叫 **加权平均时间复杂度** 或者 **期望时间复杂度**。
- **均摊时间复杂度** (amortized time complexity)：在代码执行的所有复杂度情况中绝大部分是低级别的复杂度，个别情况是高级别复杂度且发生具有时序关系时，可以将个别高级别复杂度均摊到低级别复杂度上。基本上均摊结果就等于低级别复杂度。

举例说明：

```
1 // n 表示数组 array 的长度
2 function find(array, n, x) {
3   let i = 0;
4   let pos = -1;
5   for (; i < n; ++i) {
6     if (array[i] == x) {
7       pos = i;
8       break;
9     }
10  }
11  return pos;
12 }
```

find 函数实现的功能是在一个数组中找到值等于 x 的项，并返回索引值，如果没找到就返回 -1。

最好情况时间复杂度，最坏情况时间复杂度

如果数组中第一个值就等于 x，那么时间复杂度为 $O(1)$ ，如果数组中不存在变量 x，那我们就需要把整个数组都遍历一遍，时间复杂度就成了 $O(n)$ 。所以，不同的情况下，这段代码的时间复杂度是不一样的。

所以上面代码的 **最好情况时间复杂度** 为 $O(1)$ ，**最坏情况时间复杂度** 为 $O(n)$ 。

平均情况时间复杂度

如何分析平均时间复杂度？代码在不同情况下复杂度出现量级差别，则用代码所有可能情况下执行次数的加权平均值表示。

要查找的变量 x 在数组中的位置，有 $n+1$ 种情况：在数组的 $0 \sim n-1$ 位置中和不在数组中。我们把每种情况下，查找需要遍历的元素个数累加起来，然后再除以 $n+1$ ，就可以得到需要遍历的元素个数的平均值，即：

$$\frac{1+2+3+\dots+n+n}{n+1} = \frac{n(n+3)}{2(n+1)}$$

省略掉系数、低阶、常量，所以，这个公式简化之后，得到的 **平均时间复杂度** 就是 $O(n)$ 。

我们知道，要查找的变量 x ，要么在数组里，要么就不在数组里。这两种情况对应的概率统计起来很麻烦，我们假设在数组中与不在数组中的概率都为 $1/2$ 。另外，要查找的数据出现在 $0 \sim n-1$ 这 n 个位置的概率也是一样的，为 $1/n$ 。所以，根据概率乘法法则，要查找的数据出现在 $0 \sim n-1$ 中任意位置的概率就是 $1/(2n)$ 。

因此，前面的推导过程中存在的最大问题就是，没有将各种情况发生的概率考虑进去。如果我们把每种情况发生的概率也考虑进去，那平均时间复杂度的计算过程就变成了这样：

$$1 \times \frac{1}{2n} + 2 \times \frac{1}{2n} + 3 \times \frac{1}{2n} + \dots + n \times \frac{1}{2n} + n \times \frac{1}{2}$$

$$= \frac{3n+1}{4}$$

这个值就是概率论中的 **加权平均值**，也叫 **期望值**，所以平均时间复杂度的全称应该叫 **加权平均时间复杂度** 或者 **期望时间复杂度**。

所以，根据上面结论推导出，得到的 **平均时间复杂度** 仍然是 $O(n)$ 。

均摊时间复杂度

均摊时间复杂度就是一种特殊的平均时间复杂度 (应用场景非常特殊，非常有限，这里不说)。

1.5 时间复杂度总结

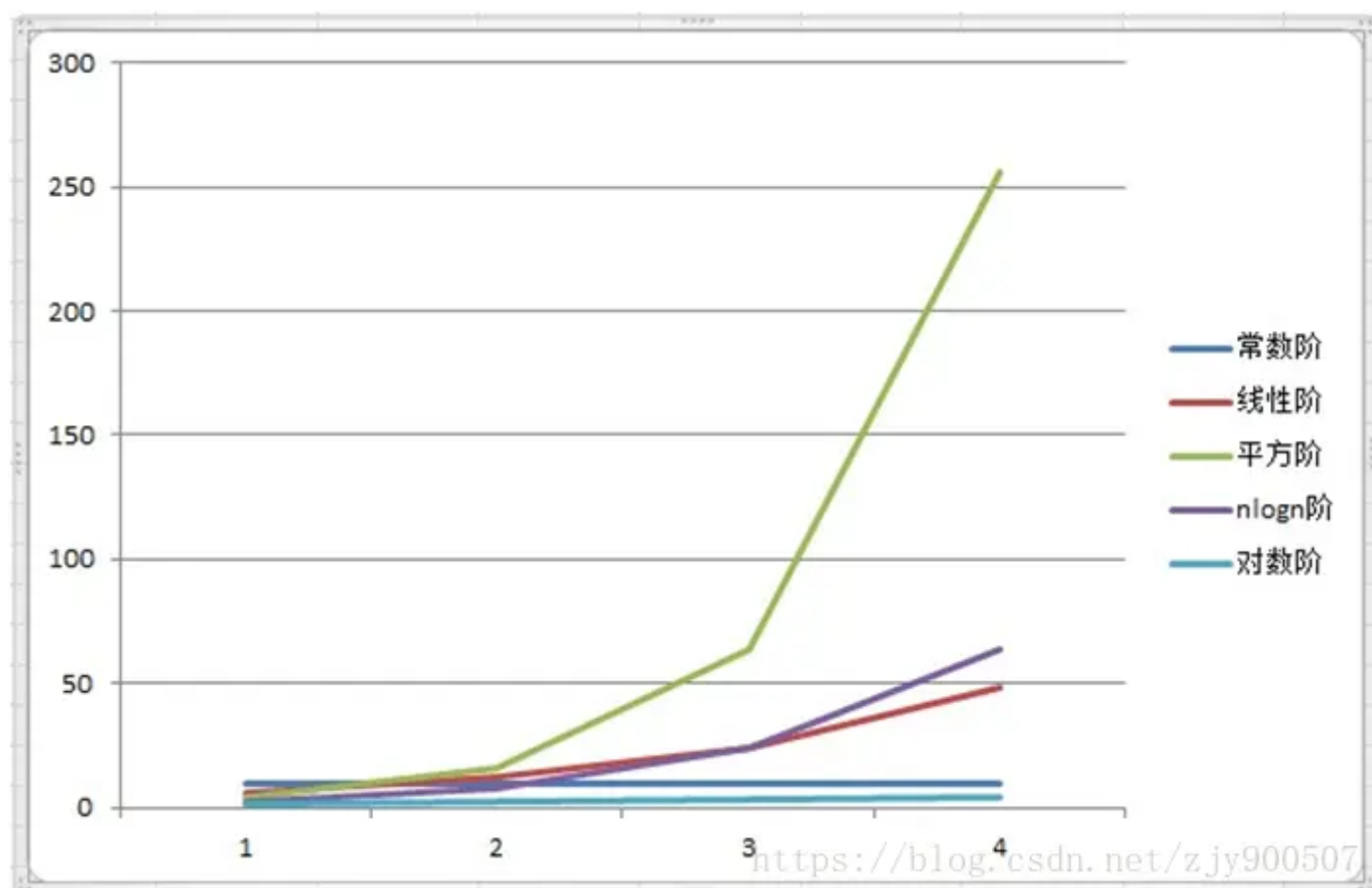
常用的时间复杂度所耗费的时间从小到大依次是：

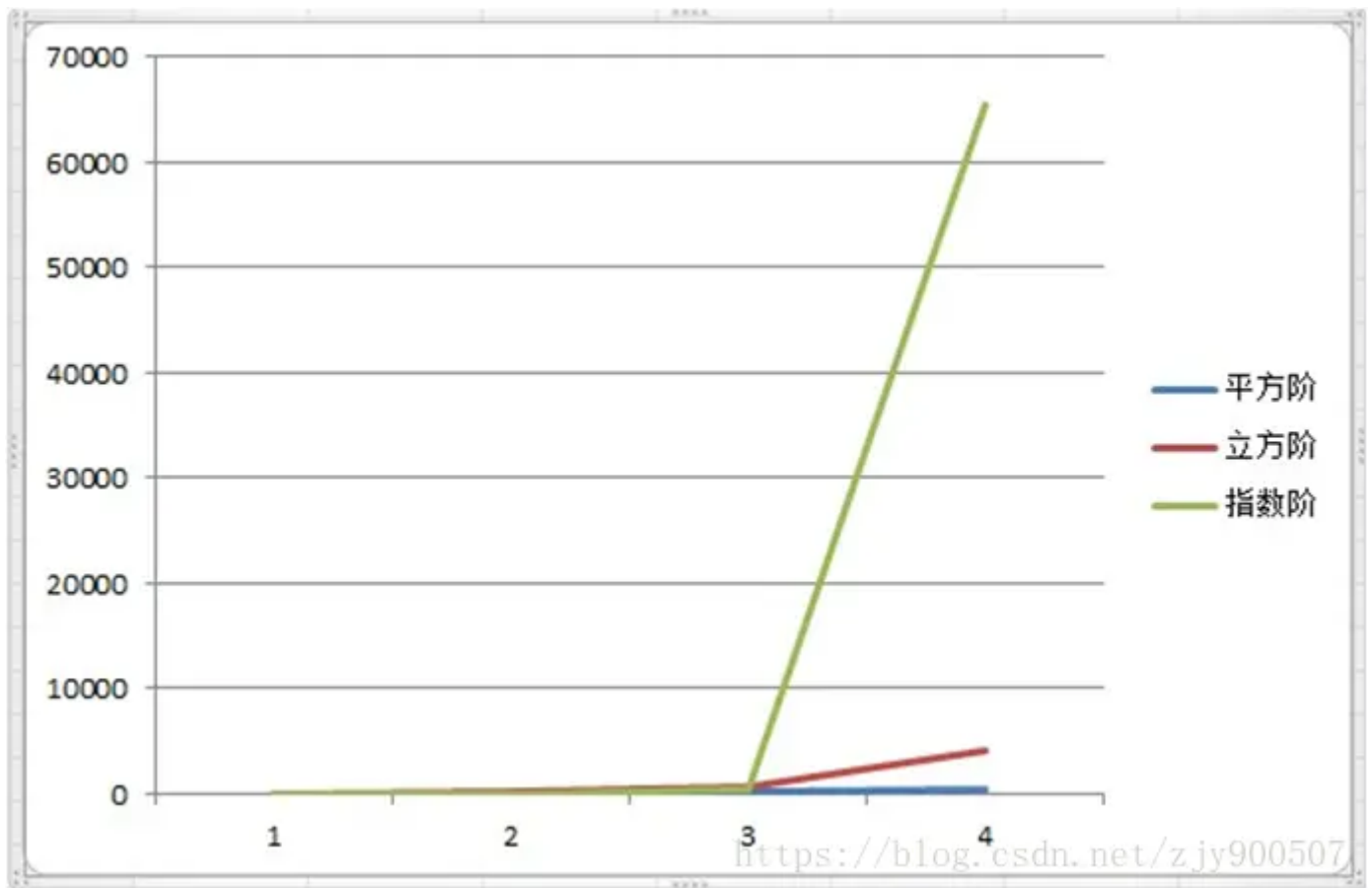
$$O(1) < O(\log n) < (n) < O(n \log n) < O(n(2)) < O(n(3)) < O(2(n)) < O(n!) < O(n(n))$$

常见的时间复杂度：

例子	时间复杂度	装逼术语
5201314	$O(1)$	常数阶
$3n+4$	$O(n)$	线性阶
$3n^2+4n+5$	$O(n^2)$	平方阶
$3\log(2)n+4$	$O(\log n)$	对数阶
$2n+3n\log(2)n+14$	$O(n\log n)$	$n\log n$ 阶
n^3+2n^2+4n+6	$O(n^3)$	立方阶
2^n	$O(2^n)$	指数阶

<https://blog.csdn.net/zjy900507>





1.6 空间复杂度分析

时间复杂度的全称是 **渐进时间复杂度**，表示 **算法的执行时间与数据规模之间的增长关系**。

类比一下，空间复杂度全称就是 **渐进空间复杂度**（asymptotic space complexity），表示 **算法的存储空间与数据规模之间的增长关系**。

定义：算法的空间复杂度通过计算算法所需的存储空间实现，算法的空间复杂度的计算公式记作：

$S(n) = O(f(n))$ ，其中， n 为问题的规模， $f(n)$ 为语句关于 n 所占存储空间的函数。

```
1 function print(n) {
2   const newArr = []; // 第 2 行
3   newArr.length = n; // 第 3 行
4   for (let i = 0; i < n; ++i) {
5     newArr[i] = i * i;
6   }
7
8   for (let j = n-1; j >= 0; --j) {
9     console.log(newArr[i])
10  }
11 }
```

跟时间复杂度分析一样，我们可以看到，第 2 行代码中，我们申请了一个空间存储变量 `newArr`，是个空数组。第 3 行把 `newArr` 的长度修改为 n 的长度的数组，每项的值为 `undefined`，除此之外，剩下

的代码都没有占用更多的空间，所以整段代码的空间复杂度就是 $O(n)$ 。

我们常见的空间复杂度就是 $O(1)$ 、 $O(n)$ 、 $O(n^2)$ ，像 $O(\log n)$ 、 $O(n \log n)$ 这样的对数阶复杂度平时都用不到。

2. 前端高频算法考点（part 1）

2.1 双指针

2.1.1 接近的三数之和

给定一个包括 n 个整数的数组 `nums` 和一个目标值 `target`。找出 `nums` 中的三个整数，使得它们的和与 `target` 最接近。返回这三个数的和。假定每组输入只存在唯一答案。

- 1 示例：
- 2
- 3 输入：`nums = [-1,2,1,-4]`，`target = 1`
- 4 输出：2
- 5 解释：与 `target` 最接近的和是 2 ($-1 + 2 + 1 = 2$)。

提示：

```
3 <= nums.length <= 10^3
-10^3 <= nums[i] <= 10^3
-10^4 <= target <= 10^4
```

链接：<https://leetcode-cn.com/problems/3sum-closest>

先按照升序排序，然后分别从左往右依次选择一个基础点 `i` ($0 \leq i \leq \text{nums.length} - 3$)，在基础点的右侧用双指针去不断的找最小的差值。

假设基础点是 `i`，初始化的时候，双指针分别是：

- **left**：`i + 1`，基础点右边一位。
- **right**：`nums.length - 1` 数组最后一位。

然后求此时的和，如果和大于 `target`，那么可以把右指针左移一位，去试试更小一点的值，反之则把左指针右移。

在这个过程中，不断更新全局的最小差值 `min`，和此时记录下来的和 `res`。

最后返回 `res` 即可。

```

2  * @param {number[]} nums
3  * @param {number} target
4  * @return {number}
5  */
6  let threeSumClosest = function (nums, target) {
7      let n = nums.length
8      if (n === 3) {
9          return getSum(nums)
10     }
11     // 先升序排序 此为解题的前置条件
12     nums.sort((a, b) => a - b)
13
14     let min = Infinity // 和 target 的最小差
15     let res
16
17     // 从左往右依次尝试定一个基础指针 右边至少再保留两位 否则无法凑成3个
18     for (let i = 0; i <= nums.length - 3; i++) {
19         let basic = nums[i]
20         let left = i + 1; // 左指针先从 i 右侧的第一位开始尝试
21         let right = n - 1 // 右指针先从数组最后一项开始尝试
22
23         while (left < right) {
24             let sum = basic + nums[left] + nums[right] // 三数求和
25             // 更新最小差
26             let diff = Math.abs(sum - target)
27             if (diff < min) {
28                 min = diff
29                 res = sum
30             }
31             if (sum < target) {
32                 // 求出的和如果小于目标值的话 可以尝试把左指针右移 扩大值
33                 left++
34             } else if (sum > target) {
35                 // 反之则右指针左移
36                 right--
37             } else {
38                 // 相等的话 差就为0 一定是答案
39                 return sum
40             }
41         }
42     }
43
44     return res
45 };
46
47 function getSum(nums) {
48     return nums.reduce((total, cur) => total + cur, 0)

```

2.1.2 通过删除字母匹配到字典里最长单词

给定一个字符串和一个字符串字典，找到字典里面最长的字符串，该字符串可以通过删除给定字符串的某些字符来得到。如果答案不止一个，返回长度最长且字典顺序最小的字符串。如果答案不存在，则返回空字符串。

```

1  示例 1:
2
3  输入:
4  s = "abpcplea", d = ["ale","apple","monkey","plea"]
5
6  输出:
7  "apple"
8  示例 2:
9
10 输入:
11 s = "abpcplea", d = ["a","b","c"]
12
13 输出:
14 "a"
15 说明:
16
17 所有输入的字符串只包含小写字母。
18 字典的大小不会超过 1000。
19 所有输入的字符串长度不会超过 1000。

```

链接: <https://leetcode-cn.com/problems/longest-word-in-dictionary-through-deleting>

为数组 `d` 中的**每个**单词分别生成**指针**，指向那个单词目前**已经匹配到**的字符下标。然后对 `s` 进行一次遍历，每轮循环中都对 `d` 中所有指针的**下一位**进行匹配，如果匹配到了，则那个单词的指针后移。一旦某个单词指针移动到那个字母的最后一位了，就和全局的 `find` 变量比较，先比较长度，后比较字典序，记录下来，最后返回即可。

```

1  /**
2   * @param {string} s
3   * @param {string[]} d
4   * @return {string}
5   */
6  let findLongestWord = function (s, d) {

```

```

7   let n = d.length
8   let points = Array(n).fill(-1)
9
10  let find = ""
11  for (let i = 0; i < s.length; i++) {
12      let char = s[i]
13      for (let j = 0; j < n; j++) {
14          let targetChar = d[j][points[j] + 1]
15          if (char === targetChar) {
16              points[j]++
17              let word = d[j]
18              let wl = d[j].length
19              if (points[j] === wl - 1) {
20                  let fl = find.length
21                  if (wl > fl || (wl === fl && word < find)) {
22                      find = word
23                  }
24              }
25          }
26      }
27  }
28
29  return find
30 }

```

2.1.3 搜索二维矩阵

编写一个高效的算法来搜索 $m \times n$ 矩阵 matrix 中的一个目标值 target。该矩阵具有以下特性：

每行的元素从左到右升序排列。

每列的元素从上到下升序排列。

示例：

现有矩阵 matrix 如下：

```

1  [
2    [1, 4, 7, 11, 15],
3    [2, 5, 8, 12, 19],
4    [3, 6, 9, 16, 22],
5    [10, 13, 14, 17, 24],
6    [18, 21, 23, 26, 30],
7  ]

```

给定 target = 5，返回 true。

给定 `target = 20`，返回 `false`。

链接：<https://leetcode-cn.com/problems/search-a-2d-matrix-ii>

由题可得，当某一个单元格的值大于目标值的时候，那么目标值一定在这个单元格**上方**的行里。

如果单元格的值小于目标值的时候，那么目标值一定在这个单元格**右边**的列里。

根据这一特性，从最左下角那一格开始，设立**行指针** `row = y - 1`，设立**列指针** `column = 0`，然后不断根据上面描述特征移动这两个指针，直到找到目标值位置。如果超出边界还没有找到目标值，那么就返回 `false`。

```
1 /**
2  * @param {number[][]} matrix
3  * @param {number} target
4  * @return {boolean}
5  */
6 let searchMatrix = function (matrix, target) {
7   let y = matrix.length
8   if (!y) return false
9   let x = matrix[0].length
10
11   let row = y - 1
12   let column = 0
13   while (row >= 0 && column < x) {
14     let val = matrix[row][column]
15     if (val > target) {
16       row--
17     } else if (val < target) {
18       column++
19     } else if (val === target) {
20       return true
21     }
22   }
23   return false
24 }
```

2.1.4 判断子序列

给定字符串 `s` 和 `t`，判断 `s` 是否为 `t` 的子序列。

你可以认为 `s` 和 `t` 中仅包含英文小写字母。字符串 `t` 可能会很长（长度 $\approx 500,000$ ），而 `s` 是个短字符串（长度 ≤ 100 ）。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

```
1 示例 1:
2 s = "abc", t = "ahbgdc"
3
4 返回 true.
5
6 示例 2:
7 s = "axc", t = "ahbgdc"
8
9 返回 false.
```

后续挑战：

如果有大量输入的 S，称作 S1, S2, ..., Sk 其中 k >= 10 亿，你需要依次检查它们是否为 T 的子序列。在这种情况下，你会怎样改变代码？

链接：<https://leetcode-cn.com/problems/is-subsequence>

判断字符串 s 是否是字符串 t 的子序列，我们可以建立一个 i 指针指向「当前 s 已经在 t 中成功匹配的字符下标后一位」。之后开始遍历 t 字符串，每当在 t 中发现 i 指针指向的目标字符时，就可以把 i 往后前进一位。

- 一旦 `i === t.length`，就代表 t 中的字符串全部按顺序在 s 中找到了，返回 true。
- 当遍历 s 结束后，就返回 false，因为 i 此时并没有成功走 t 的最后一位。

```
1 /**
2  * @param {string} s
3  * @param {string} t
4  * @return {boolean}
5  */
6 let isSubsequence = function (s, t) {
7     let sl = s.length
8     if (!sl) {
9         return true
10    }
11
12    let i = 0
13    for (let j = 0; j < t.length; j++) {
14        let target = s[i]
15        let cur = t[j]
16        if (cur === target) {
17            i++
18            if (i === sl) {
19                return true
20            }
21        }
22    }
```

```
22     }
23
24     return false
25 };
```

2.1.5 分发饼干

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。对每个孩子 i ，都有一个胃口值 g_i ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 s_j 。如果 $s_j \geq g_i$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

注意：

你可以假设胃口值为正。

一个小朋友最多只能拥有一块饼干。

```
1  示例 1:
2
3  输入: [1,2,3], [1,1]
4
5  输出: 1
6
7  解释:
8  你有三个孩子和两块小饼干，3个孩子的胃口值分别是：1,2,3。
9  虽然你有两块小饼干，由于他们的尺寸都是1，你只能让胃口值是1的孩子满足。
10 所以你应该输出1。
11 示例 2:
12
13 输入: [1,2], [1,2,3]
14
15 输出: 2
16
17 解释:
18 你有两个孩子和三块小饼干，2个孩子的胃口值分别是1,2。
19 你拥有的饼干数量和尺寸都足以让所有孩子满足。
20 所以你应该输出2。
```

链接：<https://leetcode-cn.com/problems/assign-cookies>

把饼干和孩子的需求都排序好，然后从最小的饼干分配给需求最小的孩子开始，不断的尝试新的饼干和新的孩子，这样能保证每个分给孩子的饼干都恰到好处的不浪费，又满足需求。

利用双指针不断的更新 i 孩子的需求下标和 j 饼干的值，直到两者有其一达到了终点位置：

1. 如果当前的饼干不满足孩子的胃口，那么把 `j++` 寻找下一个饼干，不用担心这个饼干被浪费，因为这个饼干更不可能满足下一个孩子（胃口更大）。
2. 如果满足，那么 `i++; j++; count++` 记录当前的成功数量，继续寻找下一个孩子和下一个饼干。

```
1 /**
2  * @param {number[]} g
3  * @param {number[]} s
4  * @return {number}
5  */
6 let findContentChildren = function (g, s) {
7   g.sort((a, b) => a - b)
8   s.sort((a, b) => a - b)
9
10  let i = 0
11  let j = 0
12
13  let count = 0
14  while (j < s.length && i < g.length) {
15    let need = g[i]
16    let cookie = s[j]
17
18    if (cookie >= need) {
19      count++
20      i++
21      j++
22    } else {
23      j++
24    }
25  }
26
27  return count
28 }
```

2.1.6 验证回文串

给定一个字符串，验证它是否是回文串，只考虑字母和数字字符，可以忽略字母的大小写。

说明：本题中，我们将空字符串定义为有效的回文串。

示例 1:

```
1 输入: "A man, a plan, a canal: Panama"
2 输出: true
```

示例 2:

- 1 输入: "race a car"
- 2 输出: false

<https://leetcode-cn.com/problems/valid-palindrome>

先根据题目给出的条件，通过正则把不匹配字符去掉，然后转小写。

建立双指针 `i`, `j` 分别指向头和尾，然后两个指针不断的向中间靠近，每前进一步就对比两端的字符串是否相等，如果不相等则直接返回 false。

如果直到 `i >= j` 也就是指针对撞了，都没有返回 false，那就说明符合「回文」的定义，返回 true。

```
1 /**
2  * @param {string} s
3  * @return {boolean}
4  */
5 let isPalindrome = function(s) {
6     s = s.replace(/[^0-9a-zA-Z]/g, '').toLowerCase()
7     let i = 0
8     let j = s.length - 1
9
10    while(i < j) {
11        let head = s[i]
12        let tail = s[j]
13
14        if (head !== tail) {
15            return false
16        } else {
17            i++
18            j--
19        }
20    }
21    return true
22 };
```

2.1.7 合并两个有序数组

给你两个有序整数数组 nums1 和 nums2，请你将 nums2 合并到 nums1 中，使 nums1 成为一个有序数组。

说明:

初始化 nums1 和 nums2 的元素数量分别为 m 和 n。

你可以假设 nums1 有足够的空间（空间大小大于或等于 m + n）来保存 nums2 中的元素。

示例:

输入:

```
1 nums1 = [1,2,3,0,0,0], m = 3
2 nums2 = [2,5,6],          n = 3
3
4 输出: [1,2,2,3,5,6]
```

<https://leetcode-cn.com/problems/merge-sorted-array>

2.2 滑动窗口

2.2.1 滑动窗口的最大值

给定一个数组 nums 和滑动窗口的大小 k，请找出所有滑动窗口里的最大值。

示例:

```
1 输入: nums = [1,3,-1,-3,5,3,6,7], 和 k = 3
2 输出: [3,3,5,5,6,7]
3 解释:
4
5   滑动窗口的位置                最大值
6   -----
7   [1  3  -1] -3  5  3  6  7       3
8   1 [3  -1  -3] 5  3  6  7       3
9   1  3 [-1  -3  5] 3  6  7       5
10  1  3 -1 [-3  5  3] 6  7       5
11  1  3 -1 -3 [5  3  6] 7       6
12  1  3 -1 -3  5 [3  6  7]      7
```

提示:

你可以假设 k 总是有效的，在输入数组不为空的情况下， $1 \leq k \leq$ 输入数组的大小。

链接: <https://leetcode-cn.com/problems/hua-dong-chuang-kou-de-zui-da-zhi-lcof>

滑动窗口，每次左右各滑动一位，并且求窗口中的最大值记录即可，这题坑的地方在于边界情况比较多。

```
1 let maxSlidingWindow = function (nums, k) {
2   if (k === 0 || !nums.length) {
3     return []
4   }
5   let left = 0
6   let right = k - 1
7   let res = [findMax(nums, left, right)]
8
9   while (right < nums.length - 1) {
10    right++
11    left++
12    res.push(findMax(nums, left, right))
13  }
14
15  return res
16 }
17
18 function findMax(nums, left, right) {
19   let max = -Infinity
20   for (let i = left; i <= right; i++) {
21     max = Math.max(max, nums[i])
22   }
23   return max
24 }
```

2.2.2 找到字符串中所有字母异位词

给定一个字符串 *s* 和一个非空字符串 *p*，找到 *s* 中所有是 *p* 的字母异位词的子串，返回这些子串的起始索引。

字符串只包含小写英文字母，并且字符串 *s* 和 *p* 的长度都不超过 20100。

说明：

字母异位词指字母相同，但排列不同的字符串。

不考虑答案输出的顺序。

示例 1:

```
1 输入：
2 s: "cbaebabacd" p: "abc"
```

```
3
4 输出:
5 [0, 6]
6
7 解释:
8 起始索引等于 0 的子串是 "cba", 它是 "abc" 的字母异位词。
9 起始索引等于 6 的子串是 "bac", 它是 "abc" 的字母异位词。
```

示例 2:

```
1 输入:
2 s: "abab" p: "ab"
3
4 输出:
5 [0, 1, 2]
6
7 解释:
8 起始索引等于 0 的子串是 "ab", 它是 "ab" 的字母异位词。
9 起始索引等于 1 的子串是 "ba", 它是 "ab" 的字母异位词。
10 起始索引等于 2 的子串是 "ab", 它是 "ab" 的字母异位词。
```

链接: <https://leetcode-cn.com/problems/find-all-anagrams-in-a-string>

还是典型的滑动窗口去解决的问题, 由于字母异位词一定是长度相等的, 所以我们需要把窗口的长度始终维持在目标字符串的长度, 也就是说, 每次循环结束后 `left` 和 `right` 是同步前进的。

由于字母异位词不需要考虑顺序, 所以只需要运用一个辅助函数 `isAnagrams` 去判断两个 `map` 中记录的字母次数, 即可判断出当前位置开始的子串是否和目标字符串形成字母异位词。

```
1 /**
2  * @param {string} s
3  * @param {string} p
4  * @return {number[]}
5  */
6 let findAnagrams = function (s, p) {
7   let targetMap = makeCountMap(p)
8   let sl = s.length
9   let pl = p.length
10  // [left,...right] 滑动窗口
11  let left = 0
12  let right = pl - 1
13  let windowMap = makeCountMap(s.substring(left, right + 1))
```

```

14   let res = []
15
16   while (left <= sl - pl && right < sl) {
17     if (isAnagrams(windowMap, targetMap)) {
18       res.push(left)
19     }
20     windowMap[s[left]]--
21     right++
22     left++
23     addCountToMap(windowMap, s[right])
24   }
25
26   return res
27 }
28
29 let isAnagrams = function (windowMap, targetMap) {
30   let targetKeys = Object.keys(targetMap)
31   for (let targetKey of targetKeys) {
32     if (
33       !windowMap[targetKey] ||
34       windowMap[targetKey] !== targetMap[targetKey]
35     ) {
36       return false
37     }
38   }
39   return true
40 }
41
42 function addCountToMap(map, str) {
43   if (!map[str]) {
44     map[str] = 1
45   } else {
46     map[str]++
47   }
48 }
49
50 function makeCountMap(strs) {
51   let map = {}
52   for (let i = 0; i < strs.length; i++) {
53     let letter = strs[i]
54     addCountToMap(map, letter)
55   }
56   return map
57 }

```

2.2.3 最小覆盖子串

给你一个字符串 S、一个字符串 T，请在字符串 S 里面找出：包含 T 所有字符的最小子串。

示例：

```
1 输入：S = "ADOBECODEBANC", T = "ABC"
2 输出："BANC"
```

说明：

如果 S 中不存这样的子串，则返回空字符串 ""。

如果 S 中存在这样的子串，我们保证它是唯一的答案。

<https://leetcode-cn.com/problems/minimum-window-substring>

根据目标字符串 `t` 生成一个目标 map，记录每个字符的目标值出现的次数。

然后就是维护一个滑动窗口，并且针对这个滑动窗口中的字符也生成一个 map 去记录字符出现次数。

每次循环都去对比窗口的 map 里的字符是否能覆盖目标 map 里的字符。

覆盖的意思就是，目标 map 里的每个字符在窗口 map 中出现，并且出现的次数要 $\geq$ 目标 map 中此字符出现的次数。

窗口滑动逻辑：

1. 如果当前还没有能覆盖，那么就右滑右边界。
2. 如果当前已经覆盖了，记录下当前的子串，并且右滑左边界看看能否进一步缩小子串的长度。

两种情况下停止循环，返回结果：

1. 左边界达到 给定字符长度 - 目标字符的长度，此时不管匹配与否，都是最短能满足的了。
2. 右边界超出给定字符的长度，这种情况会出现在右边界已经到头了，但是还没有能覆盖目标字符串，此时就算继续滑动也不可能得到结果了。

```
1 var findAnagrams = function(s, p) {
2   let sL = s.length, pL = p.length
3   if (sL < pL) return [];
4
5   let left = 0, right = 0, // 滑动窗口两侧索引
6       windowMap = new Map(), // 统计滑动窗口内每个 char 的数量
7       pMap = new Map(), // 统计字符串 p 内每个 char 的数量
8       valid = 0, // 滑动窗口 和 字符串 p 中数量相等的 char 的个数
9       ret = [] // 结果集
10
11   for (const char of p) {
12     pMap.set(char, (pMap.get(char) || 0) + 1)
```

```

13     }
14
15     while (right < sL) {
16         let char = s[right++]
17         if (pMap.has(char)) {
18             windowMap.set(char, (windowMap.get(char) || 0) + 1);
19             (windowMap.get(char) === pMap.get(char)) && valid++
20         }
21
22         if (right - left === pL) {
23             // 滑动窗口内的字符数和 pL 相等时, 并且 2 个 map 中每个 char 的数量都相等, 说明找到异位词了。
24             valid === pMap.size && ret.push(left)
25
26             // 无论有没有找到异位词, left 都要右移了。
27             let char = s[left++]
28             if (pMap.has(char)) {
29                 // 如果 pMap 中有 char, 并且 pMap 中 char 的数量和 windowMap 中的相等, (因为 left 要右移了, 所以) valid--,
30                 (windowMap.get(char) === pMap.get(char)) && valid--;
31                 // 因为 left 要右移了, 所以 windowMap 中的 char 数量也 - 1,
32                 windowMap.set(char, windowMap.get(char) - 1);
33             }
34         }
35     }
36     return ret
37 };

```

2.2.4 无重复字符的最长子串

给定一个字符串, 请你找出其中不含有重复字符的 最长子串 的长度。

示例 1:

- 1 输入: "abcabcbb"
- 2 输出: 3
- 3 解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。

示例 2:

- 1 输入: "bbbbbb"
- 2 输出: 1
- 3 解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。

示例 3:

- 1 输入: "pwwkew"
- 2 输出: 3
- 3 解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。
- 4 请注意, 你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。

链接: <https://leetcode-cn.com/problems/longest-substring-without-repeating-characters>

这题是比较典型的滑动窗口问题, 定义一个左边界 `left` 和一个右边界 `right`, 形成一个窗口, 并且在这个窗口中保证不出现重复的字符串。

这需要用到一个新的变量 `freqMap`, 用来记录窗口中的字母出现的频率数。在此基础上, 先尝试取窗口的右边界再右边一个位置的值, 也就是 `str[right + 1]`, 然后拿这个值去 `freqMap` 中查找:

1. 这个值没有出现过, 那就直接把 `right ++`, 扩大窗口右边界。
2. 如果这个值出现过, 那么把 `left ++`, 缩进左边界, 并且记得把 `str[left]` 位置的值在 `freqMap` 中减掉。

循环条件是 `left < str.length`, 允许左边界一直滑动到字符串的右界。

```
1 /**
2  * @param {string} s
3  * @return {number}
4  */
5 let lengthOfLongestSubstring = function (str) {
6   let n = str.length
7   // 滑动窗口为s[left...right]
8   let left = 0
9   let right = -1
10  let freqMap = {} // 记录当前子串中下标对应的出现频率
11  let max = 0 // 找到的满足条件子串的最长长度
12
13  while (left < n) {
14    let nextLetter = str[right + 1]
15    if (!freqMap[nextLetter] && nextLetter !== undefined) {
16      freqMap[nextLetter] = 1
17      right++
18    } else {
19      freqMap[str[left]] = 0
20      left++
21    }
22  }
```

```
21     }
22     max = Math.max(max, right - left + 1)
23 }
24
25 return max
26 }
```

3. 现代前端工程化所需完整技术图谱

 **EncodeStudio**
印客学院

| 印客学院2023匠心之作
内容&服务全面升级

Web前端 大厂工程师训练营

 印客2023大厂前端工程师课程大纲.pdf

4. 前端高频算法考点（part 2）

4.1 图

4.1.1 找到小镇的法官

在一个小镇里，按从 1 到 N 标记了 N 个人。传言称，这些人中有一个是小镇上的秘密法官。
如果小镇的法官真的存在，那么：

1. 小镇的法官不相信任何人。
2. 每个人（除了小镇法官外）都信任小镇的法官。
3. 只有一个人同时满足属性 1 和属性 2。

给定数组 `trust`，该数组由信任对 `trust[i] = [a, b]` 组成，表示标记为 `a` 的人信任标记为 `b` 的人。

如果小镇存在秘密法官并且可以确定他的身份，请返回该法官的标记。否则，返回 `-1`。

示例 1:

输入: `N = 2, trust = [[1,2]]` 输出: `2`

示例 2:

输入: `N = 3, trust = [[1,3],[2,3]]` 输出: `3`

示例 3:

输入: `N = 3, trust = [[1,3],[2,3],[3,1]]` 输出: `-1`

示例 4:

示例 4:

输入: `N = 3, trust = [[1,2],[2,3]]` 输出: `-1`

示例 5:

输入: `N = 4, trust = [[1,3],[1,4],[2,3],[2,4],[4,3]]` 输出: `3`

提示:

- `1 <= N <= 1000`
- `trust.length <= 10000`
- `trust[i]` 是完全不同的
- `trust[i][0] != trust[i][1]`
- `1 <= trust[i][0], trust[i][1] <= N`

leetcode地址: [leetcode](#)

本题是一道典型的有向图问题。寻找法官即是寻找 入度为N-1，出度为0的点

```
1 let findJudge = function(N, trust) {
2   //构造0-N个节点的图
3   let graph = Array.from({length:N+1}, () => ({outDegree:0, inDegree:0}))
4   trust.forEach(([a, b]) => {
5     graph[a].outDegree++
6     graph[b].inDegree++
7   })
8   return graph.findIndex(({outDegree, inDegree}, index) => {
9     //剔除0
10    return index != 0 && outDegree === 0 && inDegree === N-1
11  })
12 }
```

```
11    })
12  };
```

时间复杂度： $O(N)$

空间复杂度： $O(N)$

另一种解法：

其实法官也是唯一一个 **出入度差值为 $N-1$** ，所以这里可以简化为寻找出入度差值为 $N-1$

```
1  let findJudge = function(N, trust) {
2    let graph = Array(N+1).fill(0)
3    // 出度加一
4    // 入度减一
5    trust.forEach(([a, b]) => {
6      graph[a]--
7      graph[b]++
8    })
9    return graph.findIndex((node, index) => {
10     // 剔除0
11     return index !== 0 && node === N-1
12   })
13  };
```

时间复杂度： $O(N)$

空间复杂度： $O(N)$

4.1.2 课程表问题

你这个学期必须选修 `numCourse` 门课程，记为 `0` 到 `numCourse-1`。

在选修某些课程之前需要一些先修课程。例如，想要学习课程 `0`，你需要先完成课程 `1`，我们用一个匹配来表示他们：`[0,1]`

给定课程总量以及它们的先决条件，请你判断是否可能完成所有课程的学习？

示例 1:

输入: 2, [[1,0]]

输出: true

解释: 总共有 2 门课程。学习课程 1 之前，你需要完成课程 0。所以这是可能的。

示例 2:

输入: 2, [[1,0],[0,1]]

输出: false

解释: 总共有 2 门课程。学习课程 1 之前，你需要先完成课程 0；并且学习课程 0 之前，你还应先完成课程 1。这是不可能的。

提示：

- 输入的先决条件是由 **边缘列表** 表示的图形，而不是 邻接矩阵
- 你可以假定输入的先决条件中没有重复的边
- `1 <= numCourses <= 10^5`

leetcode地址：[leetcode](#)

这是一道经典的拓扑排序问题，所谓拓扑排序，举个例子：

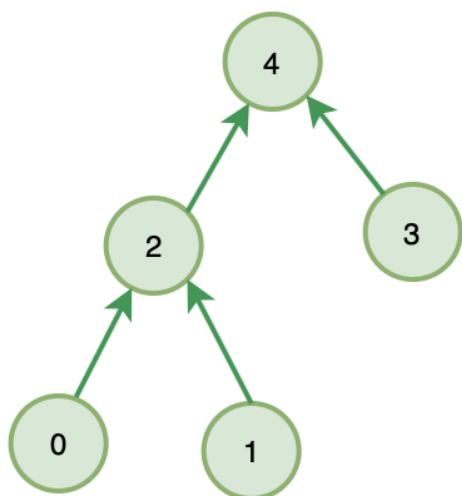
比如吃自助火锅，有一套约定俗成的流程，首先先打开包装，然后放入粉、佐料、菜，然后在加水，最后盖上盖子；这是有一套先后顺序的，你不可能没打开包装就放佐料，也可以说这是有一套依赖关系的，盖盖子依赖加水，加水依赖放入粉、佐料、菜，继而依赖打开包装

这种关系通常使用有向图来表示，如果这套流程能够成功的帮助你最后吃到火锅（无环），那这种依赖顺序就是拓扑排序，即拓扑排序是针对有向无环图的

解法：广度优先遍历

例如：

```
1 输入：5, [[4,2],[4,3],[2,0],[2,1]]
```



学习4课程前，需要先完成课程2和3

学习2课程前，需要先完成课程0和1

其中，0、1、3的入度为0，2、4的入度为1

所以我们可以使用 **邻接表** 来表示有向图中各个节点的依赖关系，同时维护一个入度表，则入度表中入度为 0 的节点所表示的课程是可以立即开始学习的（没有先决条件或先决条件已完成）

那么这题就很简单了：

- 创建一个队列，并将邻接表中所有入度为 0 的节点放入队列中
- 若队列非空，则从队列中出队第一个节点，`numCourse - 1`（学习该课程），然后将依赖该课程所有邻接节点的入度减 1
 - 若减 1 后节点入度为 0，则该课程又是可立即学习课程，将该节点添加到队尾
 - 若减 1 后节点入度不为 0，则继续遍历下一节点
- 当队列为空，检查 `numCourses === 0`（所有课程是否全部学习结束）即可

```
1 let canFinish = function(numCourses, prerequisites) {
2   // 如果没有先决条件，即所有的课程均没有依赖关系
3   // 直接返回 true
4   if (prerequisites.length === 0) {
5     return true
6   }
7
8   // 维护入度表
9   let inDegree = new Array(numCourses).fill(0)
10  // 维护邻接表
11  let adj = new Map()
12
13  for (let e of prerequisites) {
14    inDegree[e[0]]++
```

```

15         if(!adj.has(e[1])) adj.set(e[1], [])
16         let vEdge = adj.get(e[1])
17         vEdge.push(e[0])
18     }
19
20     let queue = []
21     // 首先加入入度为 0 的结点
22     for (let i = 0; i < numCourses; i++) {
23         if (inDegree[i] === 0) {
24             queue.push(i)
25         }
26     }
27
28     while (queue.length > 0) {
29         // 从队首移除
30         var v = queue.shift()
31         // 出队一门课程
32         numCourses--
33         if(!adj.has(v)) continue
34         // 遍历当前出队结点的所有临接结点
35         for(let w of adj.get(v)) {
36             inDegree[w]--
37             if (inDegree[w] === 0) {
38                 queue.push(w)
39             }
40         }
41     }
42     return numCourses === 0
43 }

```

复杂度分析：

- 时间复杂度： $O(V+E)$ ，遍历一个图需要访问所有节点和边
- 空间复杂度： $O(V+E)$ ，用于存储临接表等

4.2 动态规划

4.2.1 爬楼梯问题

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

注意： 给定 n 是一个正整数。

示例 1：

输入：2 输出：2 解释：有两种方法可以爬到楼顶。1. 1 阶 + 1 阶 2. 2 阶

示例 2:

输入: 3 输出: 3 解释: 有三种方法可以爬到楼顶。1. 1 阶 + 1 阶 + 1 阶 2. 1 阶 + 2 阶 3. 2 阶 + 1 阶

leetcode地址: [leetcode](https://leetcode.com/problems/climbing-stairs/)

解法: 动态规划

动态规划 (Dynamic Programming, DP) 是一种将复杂问题分解成小问题求解的策略, 但与分治算法不同的是, 分治算法要求各子问题是相互独立的, 而动态规划各子问题是相互关联的。

分治, 顾名思义, 就是分而治之, 将一个复杂的问题, 分成两个或多个相似的子问题, 在把子问题分成更小的子问题, 直到更小的子问题可以简单求解, 求解子问题, 则原问题的解则为子问题解的合并。

我们使用动态规划求解问题时, 需要遵循以下几个重要步骤:

- 定义子问题
- 实现需要反复执行解决的子问题部分
- 识别并求解出边界条件

第一步: 定义子问题

如果用 `dp[n]` 表示第 `n` 级台阶的方案数, 并且由题目知: 最后一步可能迈 2 个台阶, 也可迈 1 个台阶, 即第 `n` 级台阶的方案数等于第 `n-1` 级台阶的方案数加上第 `n-2` 级台阶的方案数

第二步: 实现需要反复执行解决的子问题部分

```
1 dp[n] = dp[n-1] + dp[n-2]
```

第三步: 识别并求解出边界条件

```
1 // 第 0 级 1 种方案
2 dp[0]=1
3 // 第 1 级也是 1 种方案
4 dp[1]=1
```

最后一步: 把尾码翻译成代码, 处理一些边界情况

```
1 let climbStairs = function(n) {
2     let dp = [1, 1]
```

```

3     for(let i = 2; i <= n; i++) {
4         dp[i] = dp[i - 1] + dp[i - 2]
5     }
6     return dp[n]
7 }

```

复杂度分析：

- 时间复杂度：O(n)
- 空间复杂度：O(n)

优化空间复杂度：

```

1 let climbStairs = function(n) {
2     let res = 1, n1 = 1, n2 = 1
3     for(let i = 2; i <= n; i++) {
4         res = n1 + n2
5         n1 = n2
6         n2 = res
7     }
8     return res
9 }

```

空间复杂度：O(1)

4.2.2 使用最小花费爬楼梯

数组的每个索引作为一个阶梯，第 i 个阶梯对应着一个非负数的体力花费值 `cost[i]`（索引从0开始）。

每当你爬上一个阶梯你都要花费对应的体力花费值，然后你可以选择继续爬一个阶梯或者爬两个阶梯。

您需要找到达到楼层顶部的最低花费。在开始时，你可以选择从索引为 0 或 1 的元素作为初始阶梯。

示例 1:

输入: `cost = [10, 15, 20]`

输出: 15

解释: 最低花费是从`cost[1]`开始，然后走两步即可到阶梯顶，一共花费15。

示例 2:

输入: `cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]`

输出: 6

解释: 最低花费方式是从`cost[0]`开始，逐个经过那些1，跳过`cost[3]`，一共花费6。

注意：

- `cost` 的长度将会在 `[2, 1000]` 。
- 每一个 `cost[i]` 将会是一个Integer类型，范围为 `[0, 999]` 。

leetcode地址：[leetcode](#)

本题注意理解题意：

- 第 `i` 级台阶是第 `i-1` 级台阶的阶梯顶部。
- 踏上第 `i` 级台阶花费 `cost[i]`，直接迈一大步跨过而不踏上去则不用花费。
- 楼梯顶部在数组之外，如果数组长度为 `len`，那么楼顶就在下标为 `len`

第一步：定义子问题

踏上第 `i` 级台阶的体力消耗为到达前两个阶梯的最小体力消耗加上本层体力消耗：

- 最后迈 1 步踏上第 `i` 级台阶：`dp[i-1] + cost[i]`
- 最后迈 2 步踏上第 `i` 级台阶：`dp[i-2] + cost[i]`

第二步：实现需要反复执行解决的子问题部分

所以踏上第 `i` 级台阶的最小花费为：

```
1 dp[i] = min(dp[i-2], dp[i-1]) + cost[i]
```

第三步：识别并求解出边界条件

```
1 // 第 0 级 cost[0] 种方案
2 dp[0] = cost[0]
3 // 第 1 级，有两种情况
4 // 1: 分别踏上第0级与第1级台阶，花费cost[0] + cost[1]
5 // 2: 直接从地面开始迈两步直接踏上第1级台阶，花费cost[1]
6 dp[1] = min(cost[0] + cost[1], cost[1]) = cost[1]
```

最后一步：把尾码翻译成代码，处理一些边界情况

```
1 let minCostClimbingStairs = function(cost) {
2   cost.push(0)
3   let dp = [], n = cost.length
```

```

4     dp[0] = cost[0]
5     dp[1] = cost[1]
6     for(let i = 2; i < n; i++){
7         dp[i] = Math.min(dp[i-2], dp[i-1]) + cost[i]
8     }
9     return dp[n-1]
10 }

```

复杂度分析：

- 时间复杂度：O(n)
- 空间复杂度：O(n)

优化：

```

1 let minCostClimbingStairs = function(cost) {
2     let n = cost.length,
3         n1 = cost[0],
4         n2 = cost[1]
5     for(let i = 2; i < n; i++){
6         let tmp = n2
7         n2 = Math.min(n1, n2) + cost[i]
8         n1 = tmp
9     }
10    return Math.min(n1, n2)
11 };

```

- 时间复杂度：O(n)
- 空间复杂度：O(1)

4.2.3 最大子序和

给定一个整数数组 `nums`，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

示例：

输入: [-2,1,-3,4,-1,2,1,-5,4]

输出: 6

解释: 连续子数组 [4,-1,2,1] 的和最大，为 6。

进阶：

如果你已经实现复杂度为 O(n) 的解法，尝试使用更为精妙的分治法求解。

leetcode地址：[leetcode](https://leetcode.com/problems/maximum-subarray/)

第一步：定义子问题

动态规划是将整个数组归纳考虑，假设我们已经知道了以第 $i-1$ 个数结尾的连续子数组的最大和 $dp[i-1]$ ，显然以第 i 个数结尾的连续子数组的最大和的可能取值要么为 $dp[i-1]+nums[i]$ ，要么就是 $nums[i]$ 单独成一组，也就是 $nums[i]$ ，在这两个数中我们取最大值

第二步：实现需要反复执行解决的子问题部分

```
1 dp[n] = Math.max(dp[n-1]+nums[n], nums[n])
```

第三步：识别并求解出边界条件

```
1 dp[0]=nums[0]
```

最后一步：把尾码翻译成代码，处理一些边界情况

因为我们在计算 $dp[i]$ 的时候，只关心 $dp[i-1]$ 与 $nums[i]$ ，因此不用把整个 dp 数组保存下来，只需设置一个 pre 保存 $dp[i-1]$ 就好了。

代码实现（优化）：

```
1 let maxSubArray = function(nums) {
2   let max = nums[0], pre = 0
3   for(const num of nums) {
4     if(pre > 0) {
5       pre += num
6     } else {
7       pre = num
8     }
9     max = Math.max(max, pre)
10  }
11  return max
12 }
```

复杂度分析：

- 时间复杂度： $O(n)$
- 空间复杂度： $O(1)$

5. 前端对算法的要求是什么？

5.1 从面试官角度分析算法面试到底面的是什么？

5.2 前端长期/突击算法面试，需要如何准备？

<https://leetcode.cn/problem-list/2ckc81c/>